MATH50011 Coursework 2: Continuous Normalizing Flows in Action and Asymptotic Confidence Intervals

02556173

March 22, 2026

Introduction

The first coursework established the theoretical foundations of continuous normalizing flows (CNFs), a class of flow-based machine learning models trained using maximum likelihood estimation (MLE). In the context of generative modeling, we saw that under this regime, probability densities evolve from a simple base to a complex target distribution via a neural ordinary differential equation (NODE), augmented by the evolving state's log-density whose expression is given by the instantaneous change of variables formula.

This second coursework is an implementation-focused continuation of the first one, with the central question no longer being how a CNF is trained, but how well its density estimation performance scales in practice with training sample size n. The metric against which this quality is measured is the expected test log-likelihood $\mu_n \coloneqq \mathbb{E}_{p^*}[\log p_{\hat{\theta}_n}(y)]$, where $\hat{\theta}_n$ is the CNF-produced MLE for θ^*, the parameter vector that makes the CNF's learned density as close to the true density p^* as possible.

Two results from the course constitute the theoretical backbone of this report. Firstly, the consistency of MLE (under mild regularity conditions) guarantees that as $n \to \infty$, $\hat{\theta}_n$ converges to the truth-approximating θ^* that minimizes the KL-divergence ("distance" between two distributions; a nonnegative quantity) [1]:

$$D_{\mathrm{KL}}\left(p^* \| p_{\hat{\theta}_n}\right) \coloneqq \int p^*(y) \log \frac{p^*(y)}{p_{\hat{\theta}_n}(y)} \, \mathrm{d}y = \mathbb{E}_{p^*}[\log p^*(y)] - \mathbb{E}_{p^*}\left[\log p_{\hat{\theta}_n}(y)\right] = -H(p^*) - \mu_n.$$

This consistency also implies that μ_n converges to the negative differential entropy $-H(p^*)$ (a measure of the "intrinsic uncertainty" and information content of the distribution p^*) in the long term, and more importantly, it allows us to quantify a "good performance" by looking at how close μ_n is to its theoretical maximum $-H(p^*)$. Secondly, the central limit theorem (CLT) and Slutsky's lemma justify constructing a 95% asymptotic confidence interval (ACI) for μ_n from a held-out test set, providing a measurement instrument with valid nominal coverage given sufficiently large test size M. Note that μ_n is analytically intractable because $\hat{\theta}_n$ is determined implicitly by a stochastic optimization procedure, and $\log p_{\hat{\theta}_n}$ itself requires solving an ODE numerically.

For the to-be-learned target distribution, we use the *banana distribution*, a volume-preserving nonlinear shear of the standard Gaussian. This choice is deliberate: unlike the two-moons or ring distributions commonly found in CNF literature, the banana distribution admits a closed-form density and tractable negative entropy $-H(p^*) = -(1 + \log 2\pi) \approx -2.8379$, providing a concrete oracle against which the ACIs can be judged. It is also non-Gaussian in shape, making it a meaningful benchmark for a CNF's expressivity.

At a glance, this report will explore

- a CNF implementation with key architectural and numerical choices based on the theoretical framework discussed in the first coursework,
- properties of the banana distribution and evaluation of the CNF's learned density at different training sample sizes, and
- a derivation of the ACI for μ_n, verifying empirically that its center rises toward the negative entropy oracle as n increases (in agreement with MLE consistency).

Implementation setup

We first summarize the CNF architecture used in this report, starting by referring back to the conceptual exploration in the first coursework where relevant derivations are given in full detail. Note that most significant parts of the code corresponding to our specific CNF implementation can be found in *Appendix*.

Our CNF models a continuous-time transformation of a base Gaussian $\mathcal{N}(0, I_2)$ via the NODE

$$\frac{\mathrm{d}y}{\mathrm{d}t}(t) = f_\theta(t, y(t)), \quad t \in [0,1], \quad y(0) \sim \mathcal{N}(0, I_2) \tag{1}$$

where the log-density of $y(t)$ evolves according to the instantaneous change of variable formula

$$\frac{\partial}{\partial t} \log p_\theta(t, y(t)) = -\operatorname{tr}\left(\frac{\partial f_\theta}{\partial y}(t, y(t))\right).$$

Integrating this and combining with the base Gaussian log-density gives a tractable negative log-likelihood (NLL) expression, which is minimized by solving the following augmented ODE backwards from $t = 1$ to $t = 0$:

$$\frac{\mathrm{d}}{\mathrm{d}t}\begin{bmatrix} y(t) \\ \log p_\theta\big(t, y(t)\big) \end{bmatrix} = \begin{bmatrix} f_\theta\big(t, y(t)\big) \\ -\operatorname{tr}\left(\frac{\partial f_\theta}{\partial y}\big(t, y(t)\big)\right) \end{bmatrix}. \tag{2}$$

In terms of the architecture of the neural network (NN) f_θ, it is implemented using ConcatSquash layers [2], where each layer computes

$$\mathrm{CS}(t, y) \coloneqq (W_1 y + b_1) \cdot \sigma(W_2 t + b_2) + W_3 t$$

with W_i and b_i being the weights and biases of three linear layers, and σ denotes the sigmoid function. This differs from a plain MLP with time concatenation, in that the sigmoid-gated multiplicative term $\sigma(W_2 t + b_2)$ allows the NN to smoothly modulate how strongly it responds to the spatial input y at each moment in the flow, and subsequently produce more stable ODE trajectories. In addition, tanh activation functions are used between hidden layers to keep f_θ smooth and bounded, but no activation is applied to the final layer to avoid contraining the flow's output in magnitude. Altogether, our NN has two hidden ConcatSquash layers of width 32.

Since the dimension in our case is $d = 2$, the trace-Jacobian $\operatorname{tr}(\partial f_\theta / \partial y)$ is computed exactly via reverse-mode autodifferentiation at $O(d^2)$ cost, not so laborious to the extent where Hutchinson's trace estimator becomes necessary to be deployed as we have seen in the first coursework.

Finally, to prevent density collapse (a pathological behavior whereby the CNF learns to "cheat" and concentrate all probability mass onto the finite training points, achieving arbitrarily high training log-likelihood at the cost of poor generalization), two regularization mechanisms are in place [2]:

1. Small Gaussian noise $\varepsilon \sim \mathcal{N}\left(0, \operatorname{diag}\left(0.05^2/\hat{\sigma}_1^2, 0.05^2/\hat{\sigma}_2^2\right)\right)$ is added to each training batch, where $\hat{\sigma}_i$ is the sample standard deviation of the i-th coordinate of the training data.
2. Weight decay is embedded via the AdamW optimizer to penalize large NN weights.

By blurring data points with noise and capping the NN's "aggression" via weight decay, these mechanisms force the model to respect the underlying shape of the data instead of just memorizing its coordinates.

Fitting banana distribution with CNFs

Before training, we establish a few important theoretical properties of our target distribution, which are used to create an oracle for assessing our CNF's performance.

As discussed above in *Introduction* alongside the non-negative quantity of KL-divergence, the theoretical maximum of the expected log-likelihood is $\sup_{\hat{\theta}_n} \mu_n = -H(p^*)$ and achieved only when the model perfectly recovers the target. It is therefore natural to declare the negative differential entropy of the true distribution as the oracle throughout our experiment.

The banana distribution in our case is defined as the pushforward of $\mathcal{N}(0, I_2)$ under the map

$$T : (X_1, X_2) \mapsto (Y_1, Y_2) = (X_1, X_2 + bX_1^2 - b)$$

with the curvature parameter being set to $b = 0.5$. The Jacobian determinant of this transformation is

$$\det \frac{\partial T}{\partial x} = \begin{vmatrix} 1 & 0 \\ 2bx_1 & 1 \end{vmatrix} \equiv 1,$$

meaning T is a volume-preserving shear. By the classic change of variable formula (which we can apply because T is trivially bijective), the density of $Y = T(X)$ satisfies

$$p^*(y) = p_X(T^{-1}(y)) \cdot \left| \det \frac{\partial T}{\partial x}(T^{-1}(y)) \right|^{-1} = p_X(T^{-1}(y)) = \frac{1}{2\pi} \exp\left(-\frac{y_1^2 + \left(y_2 - by_1^2 + b\right)^2}{2} \right)$$

where we used $X \sim \mathcal{N}(0, I_2)$ and $T^{-1}(y_1, y_2) = (y_1, y_2 - by_1^2 + b)$. This density expression will be used to plot the true density in Figure 1. Now, we have an analytic expression of banana's log-density:

$$\log p^*(y) = -\log(2\pi) - \frac{1}{2}\left(y_1^2 + \left(y_2 - by_1^2 + b\right)^2 \right).$$

This can be used to directly calculate banana's differential entropy. However, as an immediate corollary of the change of variable formula, $H(p)$ is invariant under volume-preserving bijections like T, thus more easily [3]

$$-H(p^*) = -H(\mathcal{N}(0, I_2)) = -\frac{d}{2}(1 + \log 2\pi)\Big|_{d=2} = -(1 + \log 2\pi).$$

This is precisely the value μ_n converges to and is plotted as the oracle dashed line in Figure 2.

With the target distribution and its oracle value ascertained, we now train the CNF at two representative sample sizes to visualize how well the learned density approximates p^*. Figure 1 shows the trained CNF outputs at the two training sample sizes, providing a qualitative finite-sample view of MLE consistency. The contour lines represent isodensity curves, and the blue shading visualizes the true density magnitudes: darker blue

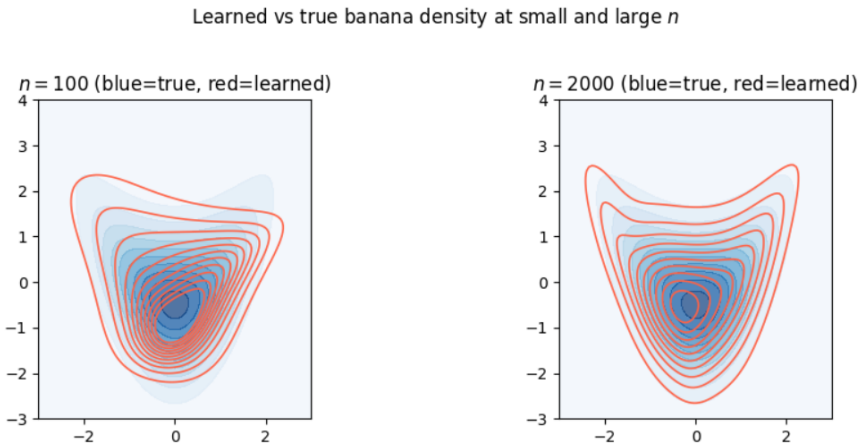


Figure 1: Comparison of banana target density and CNF learned densities at training sample sizes $n = 100$ and $n = 2000$.

indicates higher density (mass concentrated near the mode), while lighter blue indicates lower density regions toward the tails.

At the smaller training size $n = 100$, the learned red contours very roughly capture the broad banana-shaped shear but still deviate noticeably from the true blue contours in both location and curvature: the high-density core is moderately mis-centered, the inner contours are less well aligned, and the overall contour shape is considerably skewed toward the right, all of which are indications of non-negligible estimation error in $\hat{\theta}_n$ and a visible gap between $p_{\hat{\theta}_n}$ and p^*. By contrast, at $n = 2000$ the red and blue contour families are substantially closer across both inner and outer levels, showing improved recovery of the target geometry and tail behavior. This is exactly what MLE consistency predicts: as n increases, $\hat{\theta}_n \to \theta^*$, so the learned density approaches the true data-generating density, yielding increasingly similar data geometries.

Large sample behavior and ACIs

The density plots above illustrate distributional convergence qualitatively, but not a quantitative measure of how close the learned distribution is to the target. We now attempt to develop a statistical framework for this.

Fix a trained model $\hat{\theta}_n$ obtained from n training samples, and define the test log-likelihoods using samples directly from banana p^*:

$$l_j \coloneqq \log p_{\hat{\theta}_n}(y_j), \quad y \overset{iid}{\sim} p^*, \quad j = 1, \ldots, M$$

where M denotes the testing sample size.

Practically, computing these likelihoods requires solving (2), which is done in our case using an order 5 Runge-Kutta method (Tsit5; see *Appendix*). Note that they are themselves i.i.d. with mean $\mu_n = \mathbb{E}_{p^*}[\log p_{\hat{\theta}_n}(y)]$ and variance $\sigma_n^2 = \mathrm{Var}_{p^*}(\log p_{\hat{\theta}_n}(y))$, both depending on the trained CNF. Recall from *Introduction* that μ_n measures the expected quality of $\hat{\theta}_n$ as a density estimator.

By CLT, the sample mean $\bar{l}_M$ satisfies

$$\sqrt{M}\left(\bar{\ell}_M - \mu_n\right) \xrightarrow{d} \mathcal{N}(0, \sigma_n^2) \quad \text{as } M \to \infty.$$

The population variance σ_n^2 is unknown but can be consistently estimated by the sample variance

$$\hat{\sigma}_M^2 = \frac{1}{M-1} \sum_{j=1}^{M} \left(\ell_j - \bar{\ell}_M\right)^2.$$

By Slutsky, replacing σ_n with $\hat{\sigma}_M$ preserves the limiting distribution:

$$\frac{\sqrt{M}\left(\bar{\ell}_M - \mu_n\right)}{\hat{\sigma}_M} \xrightarrow{d} \mathcal{N}(0, 1).$$

This yields a 95% ACI for μ_n (using $z_{0.975} \approx 1.96$):

$$\bar{\ell}_M \pm 1.96 \, \frac{\hat{\sigma}_M}{\sqrt{M}}.$$

Two remarks are worth making here. First, this ACI is for μ_n at fixed n, so it quantifies only test sample uncertainty conditional on the trained model $\hat{\theta}_n$, not any additional variability from retraining on a new dataset. Second, while the ACI may sit below the oracle for small n due to estimation error, it tends to capture $-H(p^*)$ more reliably for sufficiently large n due to MLE asymptotics.

As seen in Figure 2, one CNF was trained independently for each $n \in \{100, 200, 500, 1000, 2000\}$ and all models were evaluated on a fixed test set of size $M = 2000$ drawn from p^*. Using a shared test set ensures that differences across intervals mainly reflect changes in μ_n rather than extra test set noise. The pattern is clear: at

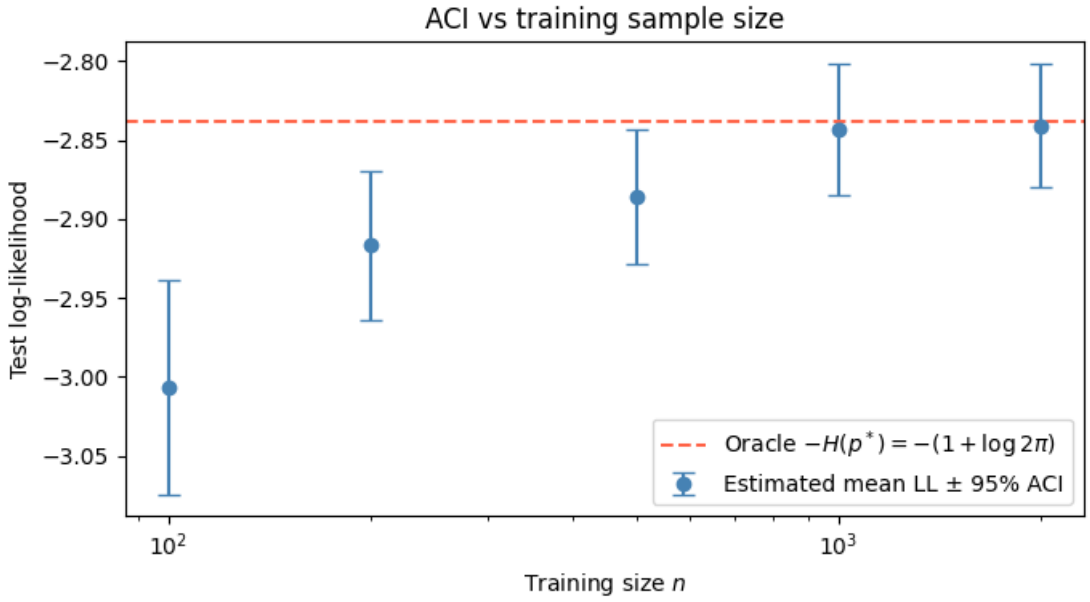


Figure 2: 95% ACIs for expected test log-likelihood μ_n versus training sample size n. The dashed red line marks the oracle value $-(1 + \log 2\pi)$.

$n = 100$, the 95% ACI lies well below the oracle $-H(p^*)$, indicating substantial finite-sample estimation error; as n increases, the interval center moves steadily upward toward the oracle, in congruence with MLE consistency. Around $n = 500$, the oracle is almost covered by the interval, and this coverage is achieved comfortably for $n = 1000$ and $n = 2000$, where likely $\mu_n \approx -H(p^*)$. Meanwhile, interval widths remain roughly stable across n, which matches the asymptotic form $1.96\,\hat{\sigma}_M/\sqrt{M}$ since M is fixed.

Why this question

This topic was chosen in the consideration that it is a natural continuation from my first coursework, allowing me to develop practical proficiency in the programming side of the subject after exploring the rich theory that connects several strands of the course with a modern machine learning paradigm. Meanwhile, I was able to consolidate core ideas from the module, such as maximum likelihood consistency and asymptotic confidence intervals. It is worth mentioning that this experience also motivates me to explore the field of neural differential equations further in the summer, in the form of UROP or otherwise.

Critical use of AI tools

AI tools were used sparingly to improve wording clarity and grammar, and partially in debugging code, while all mathematical derivations, coding decisions, and final interpretations were independently completed and verified.

Conclusion

This report investigates how the performance of CNFs scales with training sample size, using expected log-likelihood as a measure and the negative differential entropy as a theoretical benchmark. The empirical results align closely with the theory, as seen in Figure 1 and Figure 2. The use of the analytically tractable banana

distribution allows for a direct comparison between theory and experiment, reinforcing the validity of our framework. Overall, this study demonstrates that CNFs not only provide flexible density estimation in practice but also exhibit well-understood asymptotic behavior thanks to MLE as its training mechanism, making them a rather favorable framework in generative modeling. A natural direction for future work is to extend the analysis to higher-dimensional and more complex target distributions, where computational constraints necessitate techniques such as Hutchinson's trace estimator.

References

[1] Yunfan Jiang. *ELBO — What & Why*. Blog post, January 11, 2021. `https://yunfanj.com/blog/2021/01/11/ELBO.html`. Accessed March 20, 2026.

[2] Patrick Kidger. *Continuous normalising flow*. Diffrax documentation. `https://docs.kidger.site/diffrax/examples/continuous_normalising_flow/`. Accessed March 18, 2026.

[3] Gregory Gundersen. *Entropy of the Gaussian*. Blog post, September 1, 2020. `https://gregorygundersen.com/blog/2020/09/01/gaussian-entropy/`. Accessed March 20, 2026.

Appendix

Only the core components in implementing the CNF, such as likelihood via the instantaneous change of variables formula and the ConcatSquash layers, are included. Auxiliary code (plotting, experiment loops) is omitted for brevity. The packages used are:

```
import math
import jax
import jax.numpy as jnp
import jax.random as jr
import equinox as eqx
import diffrax
import optax
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
```

0.1 CNF mechanics

This code provides the implementation counterpart of the augmented ODE and log-density evolution introduced in *Implementation setup*, and underpins the likelihood calculations used in *Large sample behavior and asymptotic confidence intervals*.

```
def exact_logp_wrapper(t, y, args):
    y, _ = y
    func = args
    fn = lambda y: func(t, y, None)
    f, vjp_fn = jax.vjp(fn, y)
    (size,) = y.shape
    eye = jnp.eye(size)
    (dfdy,) = jax.vmap(vjp_fn)(eye)
    logp = jnp.trace(dfdy)
    return f, logp
```

```
def _single_log_likelihood(func, y):
    term = diffrax.ODETerm(exact_logp_wrapper)
    y0 = (y, 0.0)
    sol = diffrax.diffeqsolve(
        term, diffrax.Tsit5(),
        t0=1.0, t1=0.0, dt0=-0.05,
        y0=y0, args=func,
        saveat=diffrax.SaveAt(t1=True),
        stepsize_controller=diffrax.PIDController(rtol=1e-5, atol=1e-5),
    )
    y_out   = sol.ys[0][0]
    delta_logp = sol.ys[1][0]
    return normal_log_likelihood(y_out) + delta_logp
```

0.2 Network architecture

This code implements the ConcatSquash vector field f_θ described in *Implementation setup*, whose expressivity supports the density fits discussed in *Fitting banana distribution with CNF*.

```
class ConcatSquash(eqx.Module):
    lin1: eqx.nn.Linear
    lin2: eqx.nn.Linear
    lin3: eqx.nn.Linear

    def __init__(self, in_size, out_size, *, key):
        k1, k2, k3 = jr.split(key, 3)
        self.lin1 = eqx.nn.Linear(in_size, out_size, key=k1)
        self.lin2 = eqx.nn.Linear(1, out_size, key=k2)
        self.lin3 = eqx.nn.Linear(1, out_size, use_bias=False, key=k3)

    def __call__(self, t, y):
        return (self.lin1(y) * jax.nn.sigmoid(self.lin2(t))
                + self.lin3(t))


class Func(eqx.Module):
    layers: list

    def __init__(self, data_size, width_size, depth, *, key):
        keys = jr.split(key, depth + 1)
        layers = [ConcatSquash(data_size, width_size, key=keys[0])]
        for i in range(depth - 1):
            layers.append(ConcatSquash(width_size, width_size, key=keys[i+1]))
        layers.append(ConcatSquash(width_size, data_size, key=keys[-1]))
        self.layers = layers

    def __call__(self, t, y, args):
        t = jnp.asarray(t)[None]
        for layer in self.layers[:-1]:
            y = jax.nn.tanh(layer(t, y))
        return self.layers[-1](t, y)
```

0.3 Training

This code implements the optimization pipeline described in *Implementation setup*, and generates the fitted models used for the comparisons in *Fitting banana distribution with CNF* and Figure 2.

```python
def train_cnf(n_train, key, *, width=32, depth=2, lr=1e-3, weight_decay=1e-5):
    n_steps = {100: 400, 200: 600, 500: 1000, 1000: 1500, 2000: 2500}.get(n_train, 1000)
    batch_size = min(256, n_train)

    key, k_data, k_model = jr.split(key, 3)
    data = sample_banana(k_data, n_train)
    std  = jnp.std(data, axis=0) + 1e-6   # for noise scaling

    func = Func(data_size=2, width_size=width, depth=depth, key=k_model)

    optim = optax.adamw(lr, weight_decay=weight_decay)
    opt_state = optim.init(eqx.filter(func, eqx.is_inexact_array))

    @eqx.filter_jit
    def make_step(func, opt_state, batch, noise_key):
        batch = batch + jr.normal(noise_key, batch.shape) * 0.05 / std

        def loss_fn(f):
            lls = jax.vmap(lambda y: _single_log_likelihood(f, y))(batch)
            return -jnp.mean(lls)

        loss_val, grads = eqx.filter_value_and_grad(loss_fn)(func)
        updates, new_opt_state = optim.update(
            grads, opt_state, eqx.filter(func, eqx.is_inexact_array)
        )
        return eqx.apply_updates(func, updates), new_opt_state, loss_val

    for step in range(n_steps):
        key, k_batch, k_noise = jr.split(key, 3)
        idx   = jr.randint(k_batch, (batch_size,), 0, n_train)
        batch = data[idx]
        func, opt_state, loss = make_step(func, opt_state, batch, k_noise)
        if (step + 1) % 100 == 0:
            print(f"  step {step+1:4d}  loss = {loss:.4f}")

    return func
```